The platform, *measured.*

A technical reference for the RigaCap signal platform — infrastructure, data pipeline, services, and the production discipline that holds them together.

SECTIONS	ENDPOINTS	SYMBOLS SCANNED	DB TABLES
12	120+	~4,000	18
Full audit	FastAPI surface	Daily, US equities	PostgreSQL 15

CONTENTS

What this document *covers.*

A complete technical audit of the RigaCap platform: the infrastructure it runs on, the data it ingests, the services that turn that data into signals, and the operational discipline that keeps the production code in lockstep with the walk-forward backtest.

01 · SYSTEM OVERVIEW

A serverless platform with a *scheduled* heartbeat.

RigaCap is a serverless, event-driven signal platform on AWS. The system ingests daily market data for ~4,000 US equities through a dual-source pipeline, generates signals through a multi-factor momentum ensemble, and delivers them to subscribers through a React dashboard, an automated email pipeline, a weekly newsletter, and a moderated social engine.

The backend runs on two specialized AWS Lambda functions — an API Lambda for HTTP traffic and a Worker Lambda for scheduled and long-running jobs — both built from a single container image and differentiated by an environment variable. FastAPI sits on top of Mangum for ASGI compatibility. PostgreSQL 15 on RDS is the system of record; S3 holds the historical price cache and the pre-computed dashboard JSON the CDN serves to subscribers.

> **Two-Lambda split.** The API Lambda is small and fast (1 GB, 30s timeout) because subscribers expect dashboard loads under two seconds. The Worker Lambda is large and patient (3 GB, 900s timeout) because daily scans, walk-forward simulations, and email runs need both memory and time. They share one image so deploys are atomic.

Request and event flow

```
# Subscriber request path
React 18 SPA  →  CloudFront  →  WAF  →  API Gateway  →  API Lambda
                                                          →  RDS / S3 (read
```

```
# Scheduled / event-driven path
EventBridge  →  Worker Lambda  →  RDS / S3 (read+write)
                               →  Step Functions (walk-forward fan-out)
                               →  Claude API (content, autopsies, optimization)
                               →  SES (email)
                               →  X / Threads / IG (social)
```

What the platform produces

- **Daily signals.** Generated by the production scanner at the close, cached as JSON on S3, fronted by CloudFront for sub-second dashboard loads.
- **Daily digest email.** Subscriber-facing summary of signals, watchlist, regime context, and any sell alerts.
- **Weekly newsletter.** "The Market, Measured" — locked-draft on Saturday, published Sunday morning.
- **Social posts.** AI-drafted from real walk-forward and live trades, gated by admin approval, auto-published across X, Threads, and Instagram.
- **Mobile push notifications.** Sell alerts and high-conviction signal events through Expo.

02 · AWS INFRASTRUCTURE

Terraform, two Lambdas, and a *quiet* RDS.

All infrastructure is managed through Terraform (`infrastructure/terraform/main.tf`) against AWS account `149218244179` in `us-east-1` . State lives in a remote S3 backend with DynamoDB locking, so plans never collide between CI and the founder's laptop.

Compute

RESOURCE	IDENTIFIER	CONFIGURATION
API Lambda	`rigacap-prod-api`	Container (ECR), 1024 MB, 30s timeout, `LAMBDA_ROLE=api` . HTTP requests only; skips pickle load entirely.
Worker Lambda	`rigacap-prod-worker`	Same image, 3008 MB, 900s timeout, `LAMBDA_ROLE=worker` . Loads the full price pickle on cold start.
API Gateway	HTTP API (v2)	Custom domain `api.rigacap.com` , behind CloudFront + WAF, HTTP/2, CORS allowlist.
ECR Repository	`rigacap-prod-api`	Mutable tags, scan-on-push, retains last five images.
Step Functions	`rigacap-prod-walk-forward`	Fans walk-forward periods out across the Worker Lambda for parallel simulation.

Edge & protection

RESOURCE	IDENTIFIER	PURPOSE
CloudFront (API)	`api.rigacap.com`	Edge CDN in front of API Gateway. Injects the `X-Origin-Verify` custom header on every origin request so direct execute-api hits are rejected by the FastAPI middleware.
CloudFront (web)	`rigacap.com` · `www.rigacap.com`	Static SPA distribution; SPA-routing 404 fallback to `/index.html`.
WAF Web ACL	`aws_wafv2_web_acl.cloudfront`	Attached to the API CloudFront distribution. Standard managed rule sets plus rate-limit on auth endpoints.

Storage

BUCKET	ACCESS	PURPOSE
`rigacap-prod-frontend-*`	Public via CloudFront	React SPA static assets (1–2 MB total).
`rigacap-prod-price-data-*`	Private (Lambda IAM only)	Price pickle, parquet shadow, dashboard JSON, chart cards, social images, circuit-breaker state.
`rigacap-prod-terraform-state-*`	Private (operator IAM)	Remote Terraform state with versioning enabled.

Database

RESOURCE	CONFIGURATION
RDS PostgreSQL 15	`db.t3.micro`, 20 GB gp2, deletion protection on, asyncpg driver.

RESOURCE	CONFIGURATION
Connection pool	Pool size 5, max overflow 10, connect timeout 3s, query timeout 5s. Lambda connects lazily on first request to stay inside the 10s init window.
Endpoint	`rigacap-prod-db-v2.csfsa4i06rux.us-east-1.rds.amazonaws.com`

Scheduled events (EventBridge)

The platform's heartbeat is a set of EventBridge rules pointing at the Worker Lambda. Each rule passes a payload that selects a handler. The list grows; the table below covers the load-bearing rules.

RULE	SCHEDULE (UTC · ET)	DESCRIPTION
Market scanner	`cron(20 20 ? * MON-FRI *)` · 4:20 PM ET	Full daily scan, signal generation, dashboard JSON export. Runs 20 minutes after the close to give Alpaca SIP time to settle.
Double-signal alerts	`cron(0 22 ? * MON-FRI *)` · 5:00 PM ET	Multi-confirmation alert email.
Daily digest	`cron(0 23 ? * MON-FRI *)` · 6:00 PM ET	Subscriber-facing daily summary email.
Strategy analysis	`cron(30 23 ? * FRI *)` · Fri 6:30 PM ET	Weekly auto-analysis of strategy performance.
Newsletter draft	`cron(0 14 ? * SAT *)` · Sat 10:00 AM ET	Generates the locked draft of "The Market, Measured" for admin review.
Newsletter publish	`cron(0 14 ? * SUN *)` · Sun 10:00 AM ET	Publishes the approved newsletter to subscribers.
Nightly walk-forward	`cron(0 1 ? * TUE-SAT *)` · 8:00 PM ET	Walk-forward simulation; feeds the social-content draft queue.

RULE	SCHEDULE (UTC · ET)	DESCRIPTION
AI social content	`cron(0 2 ? * TUE-SAT *)` · 9:00 PM ET	Generates social drafts from real trade events.
Intraday monitor	`cron(0/5 14-20 ? * MON-FRI *)` · every 5 min, market hours	HWM tracking, regime exits, sell-alert dispatch.
Ticker health	`cron(0 12 ? * MON-FRI *)` · 7:00 AM ET	Pre-market sanity sweep over the universe.
Pipeline health	`cron(30 12 * * ? *)` · 7:30 AM ET	Full pipeline-health digest with admin email.
Onboarding drip	`cron(0 14 * * ? *)` · 10:00 AM ET	Five-step new-subscriber sequence over eight days.
Publish posts	`cron(0/15 * * * ? *)` · every 15 min	Auto-publishes approved social posts whose `scheduled_for` has passed.
Post notifications	`cron(0 * * * ? *)` · hourly	T-24h and T-1h admin notifications for upcoming posts.
Pickle rebuild	`cron(0 1 ? * SUN *)` · Sat 8:00 PM ET	Weekly full pickle rebuild — catches new universe symbols.
Lambda warmer	`rate(5 minutes)`	Keeps the Worker Lambda warm.
Admin morning health	`cron(0 11 ? * MON-FRI *)` · 7:00 AM ET	Operator digest: indicator validity, signal counts, pipeline status, regime.
Nightly data hygiene	`cron(30 22 ? * MON-FRI *)` · 6:30 PM ET	Asset-ID integrity, Alpaca corp-actions poll, split auto-refetch.
Meta token refresh	`cron(0 2 ? * SUN *)` · Sun 02:00 UTC	Weekly long-lived token refresh for IG and Threads; admin email on failure.

03 · CDN, DNS & SSL

Edge in front of *everything*.

Both the static SPA and the API live behind CloudFront. The web distribution handles SPA routing; the API distribution attaches the WAF and injects the origin-verify header that closes the direct-execute-api bypass. ACM provides the certificates; Route53 provides the DNS.

CloudFront — web distribution

SETTING	VALUE
Domains	`rigacap.com` , `www.rigacap.com`
Origin	S3 website endpoint (HTTP-only)
Protocol	Redirect HTTP → HTTPS
Price class	PriceClass_100 (US, Canada, Europe)
Default TTL	1 hour, max 1 day
SPA routing	Custom error: 404 → `/index.html` (200)
SSL certificate	ACM: `rigacap.com` + `*.rigacap.com` (DNS-validated)

CloudFront — API distribution

SETTING	VALUE
Domain	`api.rigacap.com`
Origin	API Gateway HTTP API

SETTING	VALUE
WAF	`aws_wafv2_web_acl.cloudfront` attached
Origin custom header	`X-Origin-Verify: <secret>` — validated by FastAPI middleware
Caching	Disabled for authenticated routes; bypass on Authorization header

Route53 DNS

RECORD	TYPE	TARGET
`rigacap.com`	A (alias)	Web CloudFront distribution
`www.rigacap.com`	A (alias)	Web CloudFront distribution
`api.rigacap.com`	A (alias)	API CloudFront distribution

PostgreSQL 15, async, *conservative.*

The application talks to PostgreSQL 15 over the async `asyncpg` driver via SQLAlchemy 2.0 async sessions. Most tables use auto-increment integer primary keys; the user identity tables use UUIDs. Schema migrations run through a dedicated `run_migration` Lambda event so the migration can land before any model code that depends on it.

Core market data

stock_data

Daily OHLCV with computed indicators. One row per symbol per trading day.

- `id` INTEGER PK
- `symbol` VARCHAR(10) IDX
- `date` DATETIME IDX
- `open, high, low, close` FLOAT
- `volume` FLOAT
- `accum_ref, ma_50, ma_200` FLOAT

signals

Trading signals from the scanner. Lifecycle: active → executed / expired.

- `id` INTEGER PK
- `symbol` VARCHAR(10) IDX
- `signal_type` VARCHAR(10) BUY/SELL
- `price, accum_ref, pct_above_ref` FLOAT
- `volume, volume_ratio` FLOAT
- `stop_loss, profit_target` FLOAT
- `is_strong` BOOLEAN
- `status` VARCHAR(20)
- `created_at, expires_at` DATETIME

positions

Open and closed portfolio positions with trailing-stop state.

id INTEGER PK	user_id UUID FK→users
symbol VARCHAR(10) IDX	entry_date, entry_price DATETIME, FLOAT
shares, stop_loss FLOAT	profit_target, highest_price FLOAT
signal_id INTEGER FK→signals	status VARCHAR(20) open/closed

trades

Closed trade records with realized P&L.

id INTEGER PK	user_id UUID FK→users
position_id INTEGER FK→positions	symbol VARCHAR(10) IDX
entry_date, exit_date DATETIME	entry_price, exit_price FLOAT
pnl, pnl_pct FLOAT	exit_reason VARCHAR(50)

Identity, strategy, and *simulation.*

Authentication and subscriptions

users

Multi-provider OAuth-aware accounts. UUID primary keys.

`id` UUID PK

`email` VARCHAR(255) UNIQUE IDX

`password_hash` VARCHAR(255) NULLABLE

`name` VARCHAR(255)

`role` VARCHAR(20) admin/user

`google_id, apple_id` VARCHAR(255) UNIQUE

`stripe_customer_id` VARCHAR(255) UNIQUE

`is_active` BOOLEAN

`created_at, last_login` DATETIME

subscriptions

Stripe-managed subscription state. Status: trial / active / canceled / expired / past_due.

`id` UUID PK

`user_id` UUID FK→users UNIQUE

`status` VARCHAR(20)

`stripe_subscription_id` VARCHAR(255)

`stripe_price_id` VARCHAR(255)

`trial_start, trial_end` DATETIME

`current_period_start/end` DATETIME

`cancel_at_period_end` BOOLEAN

referrals

Stripe-coupon-backed give-month / get-month referral program. Eight-character codes.

`id` INTEGER PK

`referrer_user_id` UUID FK→users

`code` CHAR(8) UNIQUE IDX

`stripe_coupon_id` VARCHAR(50)

`redeemed_user_id` UUID FK→users NULLABLE

`redeemed_at, rewarded_at` DATETIME

Strategy management

strategy_definitions

Trading strategy configurations. The production strategy is the multi-factor ensemble; legacy single-factor strategies are retained for historical comparison.

id INTEGER PK
name VARCHAR(100) UNIQUE
strategy_type VARCHAR(50)
parameters TEXT (JSON)
is_active BOOLEAN IDX
source VARCHAR(50) manual/ai
is_custom BOOLEAN

strategy_evaluations · strategy_switch_history

Strategy scoring and an audit trail of every switch with before/after scores and trigger.

strategy_id FK→strategy_definitions
total_return_pct FLOAT
sharpe_ratio FLOAT
max_drawdown_pct FLOAT
recommendation_score FLOAT 0–100
trigger manual / auto_scheduled
score_before / after FLOAT
reason TEXT

Simulation, intelligence, and social

walk_forward_simulations

Walk-forward runs with aggregate results. Trades stored as JSON to fit inside the Step Functions state limit.

id INTEGER PK
start_date, end_date DATETIME
reoptimization_frequency VARCHAR(20)
total_return_pct FLOAT
sharpe_ratio FLOAT
max_drawdown_pct FLOAT
trades_json TEXT (JSON)
equity_curve_json TEXT (JSON)
status pending/completed/failed
is_daily_cache BOOLEAN IDX

model_positions

Live, walk-forward, and ghost portfolio positions. Tracks signal replay and AI autopsy.

id INTEGER PK
portfolio_type VARCHAR(20) IDX
symbol VARCHAR(10) IDX
entry_date, entry_price DATETIME, FLOAT
shares, cost_basis FLOAT
highest_price, exit_price FLOAT
exit_reason, status VARCHAR
signal_data_json TEXT (JSON)
autopsy_json TEXT (JSON)

regime_forecast_snapshots · regime_history

Daily regime forecast with the seven-regime probability vector, plus weekly regime classifications used for chart band overlays.

`snapshot_date` DATETIME UNIQUE IDX

`probabilities_json` TEXT (JSON)

`recommended_action` VARCHAR(30)

`week_start` DATE UNIQUE IDX

`current_regime` VARCHAR(30)

`outlook` stable/deteriorating/improving

`spy_close, vix_close` FLOAT

`regime` VARCHAR(30)

social_posts

AI-drafted social content with admin approval workflow. Status: draft → approved → scheduled → posted / canceled.

`id` INTEGER PK

`platform` twitter / instagram / threads

`image_s3_key` VARCHAR(500)

`ai_generated, ai_model` BOOLEAN, VARCHAR

`news_context_json` TEXT (JSON)

`reply_to_*` VARCHAR (platform-specific reply IDs)

`post_type` trade_result / missed_opp / recap / contextual_reply

`status, text_content` VARCHAR, TEXT

`scheduled_for, posted_at` DATETIME

`ai_prompt_hash` VARCHAR

`source_trade_json` TEXT (JSON)

symbol_metadata · symbol_metadata_events

Layer-2 data hygiene: one row per symbol with Alpaca asset UUID and status, plus an audit log of every corporate action detected.

`symbol` VARCHAR(10) UNIQUE IDX

`status` active / inactive / quarantined

`handler_result` VARCHAR

`asset_id` UUID

`event_type` split / merger / delisting / reuse

`detected_at` DATETIME

push_tokens

Mobile push notification tokens for Expo.

`id` INTEGER PK

`user_id` UUID FK → users

`token` VARCHAR UNIQUE

`platform` ios / android

`created_at` DATETIME

Two sources, two stores, one *truth*.

The platform ingests daily market data from two providers, stores it in two formats during the parquet migration, and exposes a single read interface everywhere downstream so the production scanner and the walk-forward simulator see exactly the same bars.

Dual-source ingestion

USE CASE	PRIMARY	FALLBACK	REASON
Daily EOD bars	Alpaca SIP	yfinance	Alpaca is faster and more reliable for bulk historical pulls.
Live intraday quotes	yfinance	Alpaca	Alpaca free tier is IEX-only; yfinance covers all exchanges.
Index data (^VIX, ^GSPC)	yfinance	—	Alpaca does not serve index symbols.
Sector ETFs	Alpaca SIP	yfinance	Normal tickers, Alpaca-compatible.

Symbol normalization handles the hyphen/dot mismatch between providers (`BRK-B` ↔ `BRK.B`). Splits are pulled with `Adjustment.SPLIT` on every ingest so the cached history is always split-adjusted — preventing the unadjusted-split artifacts that surface after large stock splits.

Storage layers

S3 pickle (current primary)

`PRICES/ALL_DATA.PKL.GZ`

Gzipped pickle: **269 MB compressed, 4,360 symbols, seven-year window.** Worker Lambda loads on cold start; API Lambda never touches it. Rebuilt on every daily scan and nightly hygiene run, with a weekly full rebuild on Saturday night to incorporate new universe symbols.

S3 parquet shadow

`PRICES/ALL_DATA.PARQUET`

Columnar zstd-compressed parquet, schema-enforced via pyarrow. Written on every scan in lockstep with the pickle. Enables partial reads (one symbol without loading the full blob) and DuckDB SQL queries.

In-memory cache

`SCANNERSERVICE.DATA_CACHE`

`Dict[str, DataFrame]` on the Worker Lambda. ~1.5–2 GB footprint when fully loaded; comfortably inside the 3008 MB ceiling.

CDN dashboard JSON

`DASHBOARD/LATEST.JSON`

Pre-computed at scan time and fronted by CloudFront with a one-hour TTL. The dashboard, the daily digest, the sell-alert dispatcher, and the intraday monitor all read from this single JSON to keep emails and the UI in lockstep.

Data integrity filters

`SCANNER._IS_DATA_QUALITY_OK()`

Rejects symbols with price-to-accumulation-reference ratios above 50× or below 0.02, single-day moves above 65% (log-return > 0.5), and gaps above ten trading days — the ticker-reuse signature. Catches unadjusted splits, stitched

DuckDB ad-hoc layer

`DATA_EXPORT_SERVICE.QUERY_PARQUET()`

Lambda-compatible SQL over the parquet shadow. Powers the `parquet_diagnose` handler for universe-wide corruption scans in seconds.

histories, and delisting artifacts in real time.

Layer-2 data hygiene

Every weekday at 6:30 PM ET the `nightly_data_hygiene` rule fires a six-step protection pass against silent data drift:

1. **Asset-ID verification.** Compares each symbol's stored Alpaca asset UUID to the current value. Any change is treated as a ticker-reuse event — auto-quarantine plus admin alert.
2. **Corp-actions poll.** Queries the Alpaca corporate-actions API for splits, dividends, spin-offs, mergers, and delistings in the last 36 hours. Hundreds of events per day is typical.
3. **Split auto-refetch.** For each detected split, fetches a clean split-adjusted history with `Adjustment.SPLIT` and overwrites the cache.
4. **Dual-store re-export.** The updated cache is written to both pickle and parquet so the two stores never diverge.
5. **Post-actions diagnose.** A DuckDB scan over the fresh parquet flags anomalies above threshold.
6. **Admin digest email.** Tiered status — auto-remediated items appear as INFO; only true anomalies escalate.

The scanner consumes the quarantine list via `symbol_metadata_service.get_excluded_symbols()` at the start of each run, with no measurable latency cost.

Parquet migration status

The pickle is durable through current scale; the parquet migration follows the four-stage plan tracked in `project_storage_migration_roadmap.md`. Stage 1 (shadow write) and Stage 2 (Amazon Linux 2023 base image) are deployed. Stage 3 is in progress — a parallel-read diff harness validates parquet against pickle for an

observation window before any consumer cuts over. The end state retires the pickle entirely.

06 · BACKEND SERVICES

Singletons, schedulers, and a *circuit* breaker.

The backend is a set of singleton service classes instantiated at module load and imported across handlers. Mangum translates ASGI calls into Lambda invocations on the API path; the Worker Lambda dispatches EventBridge payloads through a handler map.

Production strategy

The deployed strategy is the RigaCap Momentum Strategy (live since June 10, 2026): up to twenty positions at 4.5% each with inverse-volatility sizing, a 30% trailing stop, a market-regime filter, and the Cascade Guard circuit breaker. Entry requires a multi-factor gauntlet — price above DWAP × 1.05, within 3% of the 50-day high, and a top momentum-composite rank — drawn from a universe of the top 100 symbols by 60-day volume ($15 minimum price, 500k minimum daily volume). Performance figures live in the investor report and the public track-record page; they are not duplicated here.

Cascade Guard (circuit breaker)

CircuitBreakerService

`CIRCUIT_BREAKER_ENABLED=TRUE` · `CB-STATE/<PORTFOLIO_TYPE>.JSON`

Live in the production scanner. When multiple positions hit their trailing stops on the same day — the cascading-stress signature — the service pauses new entries for ten days. State persists to S3 at `s3://rigacap-prod-price-data-149218244179/cb-state/<portfolio_type>.json` so the breaker survives Lambda cold starts. Externally, this is "Cascade Guard"; internally, it is the circuit breaker. Smoke-tested via `scripts/test_circuit_breaker_smoke.py` .

Seven-regime market classifier

MarketRegimeService

`STRONG_BULL / WEAK_BULL / ROTATING_BULL / RANGE_BOUND / WEAK_BEAR / PANIC_CRASH / RECOVERY`

Daily classification using SPY indicator stack (200-day MA, breadth, VIX, term-structure proxies). Each regime modulates entry filters, position sizing, and exit thresholds. Snapshots persist to `regime_forecast_snapshots` for heatmap visualization and accuracy tracking.

Service inventory

SERVICE	RESPONSIBILITY
ScannerService	Daily market scan, signal generation, in-memory cache management.
BacktesterService	Historical backtesting, portfolio simulation, performance metrics.
WalkForwardService	Step Functions orchestration, period-by-period evaluation, AI optimization.
EnsembleSignalService	Persists ensemble signals for audit trail and email-UI consistency.
MarketRegimeService	Seven-regime classification with daily snapshot persistence.
CircuitBreakerService	Cascade Guard implementation with S3-persisted state.
EmailService	Subscriber emails: welcome, daily digest, sell alerts, password reset, onboarding drip.
NewsletterService	"The Market, Measured" weekly newsletter: locked Saturday draft, Sunday publish, archive teaser generation.
SchedulerService	EventBridge dispatch, nightly digest, social-post scheduling.

SERVICE	RESPONSIBILITY
ModelPortfolioService	Live, walk-forward, and ghost portfolio tracking; what-if calculator.
DataExportService	S3 export and import, CDN pre-computation, parquet persistence, DuckDB queries.
SymbolMetadataService	Layer-2 hygiene: asset-ID checks, corp-actions handling, quarantine list.
AIContentService	Claude API client (`claude-sonnet-4-5`) for social drafts, autopsies, and editorial copy. Template fallback if the API is unavailable.
SocialContentService	Template-based post generation; safety net for the AI service.
PostSchedulerService	Spreads drafts across optimal posting windows; T-24h and T-1h admin notifications; one-click cancel.
SocialPostingService	Multi-platform publish: X (Twitter API v2), Threads (graph.threads.net), Instagram-with-Instagram-Login (graph.instagram.com) using IG-native long-lived tokens (60-day TTL, `IGAA*` prefix).
ReplyScannerService	Multi-platform contextual reply scanner across X and Threads.
InstagramCommentService	Comment monitoring with Claude-drafted replies under admin approval.
ChartCardGenerator	1080×1350 (4:5) matplotlib chart cards in editorial brand styling.
TradeAutopsyService	Claude-powered post-mortem analysis on closed trades.
RegimeForecastService	Daily regime forecast storage and accuracy tracking.
StockUniverseService	Universe management, sector classification, ETF filtering.

SERVICE	RESPONSIBILITY
StrategyAnalyzerService	Multi-strategy comparison and performance attribution.
StrategyGeneratorService	Optuna-based parameter optimization and AI strategy generation.
AutoSwitchService	Automated strategy switching with scoring and cooldown.
HealthMonitorService	Pipeline health checks across data freshness, worker performance, database, signals, and infrastructure.
PushNotificationService	Expo push notifications for the mobile app.
DualSourceProvider	Alpaca SIP primary + yfinance fallback with health tracking and auto-failover.
ReferralService	Stripe-coupon-backed give-month / get-month program with one-click code redemption.
ThisWeekService	Computes the dashboard "This Week" rotating banner from the open position set — leader, tail, and longest-hold templates.

> Lambda cold-start mitigation. Database connections are initialized lazily on first request because the Lambda init phase has a ten-second hard ceiling. An EventBridge rule pings `/health` every five minutes to keep the Lambda warm; post-deploy, `/api/warmup` preloads S3 data into memory.

07 · API SURFACE

120+ endpoints, *auth-first.*

The FastAPI surface exceeds 120 endpoints across authentication, signals, billing, dashboard, social, admin, and public marketing routes. Counts below are approximate; treat the relevant router file as the source of truth. Anything that returns signal or portfolio data sits behind `require_valid_subscription` or the admin guard.

Authentication — `/auth` + `/auth/2fa`

METHOD	PATH	DESCRIPTION
POST	/auth/register	Email/password signup with Turnstile verification.
POST	/auth/login	Email/password login; returns JWT access + refresh tokens.
POST	/auth/refresh	Refresh access token.
GET	/auth/me	Current user profile.
POST	/auth/google	Google OAuth signup / login.
POST	/auth/apple	Apple Sign In signup / login.
POST	/auth/forgot-password	Send password reset email.
POST	/auth/reset-password	Reset password with secure token.
POST	/auth/logout	Invalidate current tokens.
PATCH	/auth/me/email-preferences	Update notification preferences.

METHOD	PATH	DESCRIPTION
POST	/auth/unsubscribe	One-click email unsubscribe (RFC 8058).
GET	/auth/referral	Get / create referral code and stats.
POST	/auth/2fa/setup	Generate TOTP secret + QR code.
POST	/auth/2fa/verify	Verify 2FA code at login.
POST	/auth/2fa/disable	Disable 2FA.
POST	/auth/2fa/regenerate-backup-codes	Generate new backup codes.

Signals — `/signals`

METHOD	PATH	AUTH
GET	/signals/scan	Subscriber — runs full market scan.
GET	/signals/latest	Subscriber.
GET	/signals/dashboard	Subscriber — comprehensive dashboard data.
GET	/signals/momentum-rankings	Subscriber.
GET	/signals/missed	Subscriber — missed opportunities.
GET	/signals/double-signals	Subscriber — multi-confirmation.
GET	/signals/watchlist	Subscriber.
GET	/signals/approaching-trigger	Subscriber.
GET	/signals/symbol/{symbol}	Subscriber.
GET	/signals/cdn-url	Subscriber — signed CDN URL.

METHOD	PATH	AUTH
`POST`	/signals/backfill	Admin.

Billing — `/billing`

METHOD	PATH	DESCRIPTION
`POST`	/billing/create-checkout	Stripe Checkout session, trial with credit card required.
`POST`	/billing/portal	Stripe Customer Portal session.
`GET`	/billing/subscription	Current subscription status.
`POST`	/billing/sync	Sync subscription state from Stripe.
`POST`	/billing/webhook	Stripe webhook receiver, signature-verified.

Dashboard editorial — `/api`

METHOD	PATH	DESCRIPTION
`GET`	/api/this-week	Computes the rotating "This Week" banner data — leader, tail, and longest-hold — from the open position set, pivoting templates so the banner never reads as a defeat when the leader is flat or negative.
`GET`	/api/market-data-status	Public freshness signal for the dashboard staleness banner.
`GET`	/api/warmup	Post-deploy warm-up route — preloads dashboard JSON.
`GET`	/api/track-record	Public: walk-forward and live track-record data backing the `/track-record` page.

METHOD	PATH	DESCRIPTION
GET	/api/newsletter/archive	Public: archive of past "Market, Measured" issues with headline plus two- to three-line breadcrumbs.

Social — `/social`

METHOD	PATH	DESCRIPTION
GET	/social/posts	List posts — filter by status, type, platform.
GET	/social/posts/{id}	Get single post details.
POST	/social/posts/compose	Create new draft post.
POST	/social/posts/{id}/approve	Approve for publishing.
POST	/social/posts/{id}/reject	Reject with reason.
POST	/social/posts/{id}/publish	Publish immediately.
POST	/social/posts/{id}/schedule	Schedule for a future date.
POST	/social/posts/{id}/cancel	Cancel a scheduled post.
POST	/social/posts/{id}/regenerate-ai	Re-generate via Claude API.
GET	/social/posts/{id}/cancel-email	One-click cancel via JWT-signed link.
GET	/social/posts/{id}/approve-email	One-click approve via JWT-signed link.
POST	/social/generate-chart/{id}	Generate the chart-card image.
GET	/social/stats	Publishing statistics.

Admin — `/admin`

Admin endpoints require both `role=admin` and an `ADMIN_EMAILS` allowlist match — double enforcement so a promoted-by-mistake user cannot reach operator surfaces. Categories include strategy management, user management, regime analysis, walk-forward simulation, backtesting, data debugging, model portfolio, trade autopsies, ghost comparison, regime forecast, and the what-if calculator.

React, Vite, Tailwind — *editorial* register.

The web client is a single-page React 18 application built with Vite, styled with Tailwind CSS configured in the editorial brand palette (paper, ink, and claret). The mobile app is a separate Expo project that consumes the same API surface for sell alerts and signal events.

Component inventory

COMPONENT	SIZE	RESPONSIBILITY
App.jsx	~170 KB	Main dashboard: portfolio, signals, charts, auth, admin time-travel.
WalkForwardSimulator.jsx	~67 KB	Configure and run simulations, equity curves, trade analysis.
SocialTab.jsx	~55 KB	Post queue, AI generation, scheduling, chart cards, publishing.
AdminDashboard.jsx	~42 KB	Strategy and user management, model portfolio, ghost comparison, regime forecast, what-if calculator, trade autopsies.
FlexibleBacktest.jsx	~39 KB	Backtest configuration, strategy comparison, exit testing.
LandingPage.jsx	~30 KB	Marketing surface: hero, features, pricing, FAQ, social proof.
TrackRecordPage.jsx	~16 KB	Public `/track-record` : continuous walk-forward backtest results plus the live track record.

COMPONENT	SIZE	RESPONSIBILITY
NewsletterArchive.jsx	~12 KB	Newsletter archive teasers with headline and breadcrumb lines.
StrategyEditor.jsx	~16 KB	Custom strategy creation and editing.
AutoSwitchConfig.jsx	~15 KB	Auto-switch rule configuration.
LoginModal.jsx	~14 KB	Multi-provider auth with Turnstile.
StrategyGenerator.jsx	~12 KB	AI strategy generation interface.
LegalPages.jsx	~11 KB	Terms, Privacy Policy, Financial Disclaimer.
PasswordReset.jsx	~9 KB	Forgot / reset flow.
SubscriptionBanner.jsx	~9 KB	Trial countdown, upgrade CTA.
ThisWeekBanner.jsx	~8 KB	Rotating dashboard banner with five editorial templates pivoting on portfolio state.
TrackRecordChart.jsx	~8 KB	Walk-forward equity curve with regime overlays.
TwoFactorSettings.jsx	~7 KB	TOTP 2FA setup, backup codes.
CookieConsent.jsx	~3 KB	GDPR cookie consent for GA4.

Frontend libraries

- React 18 with hooks; functional components throughout.
- Vite 5 build tool; fast HMR in dev, single-bundle production output.
- Tailwind CSS 3 styling; brand palette tokens in `tailwind.config.js`.

- **Recharts** for the dashboard charts and equity curves.
- **Axios** HTTP client with a JWT request interceptor and a 401 retry hook.
- **React Router v6** for SPA routing.
- **Lucide React** SVG icon library.
- **react-hot-toast**, **date-fns**, **clsx** for UX plumbing.
- **Puppeteer** for headless chart rendering at the build step.

09 · SECURITY ARCHITECTURE

Defense in *depth*, not in marketing.

Every signal endpoint sits behind a subscription guard. Every admin endpoint sits behind two guards. The price-data S3 bucket has all public access blocked. The execute-api URL is no longer reachable directly. None of this is novel; all of it is enforced.

Authentication flow

```
# Subscriber login
Email / Google / Apple  →  Turnstile (bot guard)
                        →  JWT issue (HS256, 15 min access)
                        →  Bearer header on every request
                        →  Role check: admin / user
                        →  Subscription check: trial / active
```

Controls

LAYER	IMPLEMENTATION
Password hashing	bcrypt via passlib with auto-tuned cost factor.
JWT tokens	HS256, 15-minute access / seven-day refresh, python-jose.
OAuth validation	Google: ID token signature against published keys. Apple: JWKS with one-hour cache.
Bot protection	Cloudflare Turnstile on registration.

LAYER	IMPLEMENTATION
CORS	Allowlist: `rigacap.com`, `www.rigacap.com`, plus localhost dev.
Admin guard	Role plus `ADMIN_EMAILS` allowlist — double enforcement.
Subscription gate	`require_valid_subscription` on every signal and portfolio endpoint.
Payment security	Stripe Checkout (PCI DSS-validated) with webhook signature verification.
S3 access	Price-data bucket: all public access blocked, IAM-only.
HTTPS	ACM certificates everywhere; CloudFront redirects HTTP → HTTPS.
WAF	Web ACL on the API CloudFront distribution with managed rule sets and rate limits on auth.
Origin verify	CloudFront injects `X-Origin-Verify` on every API origin request. `backend/app/core/origin_guard.OriginVerifyMiddleware` compares the inbound header with `hmac.compare_digest`; direct hits to the raw execute-api URL return 403. Closes the bypass that previously made the WAF optional.
Email cancel links	JWT-signed one-click cancel and approve URLs scoped to a single post ID.

Review cadence

A security audit runs every two to three weeks. The reviewer verifies that every signal, dashboard, and portfolio endpoint requires auth; that the price-data bucket remains private; that no signal data leaks through the CDN or a public S3 path; that the CORS allowlist matches only the production origins plus dev; and that any new endpoints since the last review have proper guards in place.

Push to main, the rest is *plumbing*.

Every change ships through GitHub Actions on push to `main`. The pipeline builds the frontend, builds the Lambda container image, deploys to ECR, updates both Lambdas to the same image, and warms the API. Manual deploys are reserved for CI outages.

```
# Trigger: push to main OR pull request to main

Stage 1: Test
  Backend  → Python 3.11 → pip install → pytest -v
  Frontend → Node 18     → npm ci      → npm run lint && npm test

Stage 2: Deploy  (only on main push)
  1. npm run build → S3 sync → CloudFront invalidation
  2. Pre-deploy: export price data to S3 (pickle + parquet)
  3. Docker buildx --platform linux/amd64 --provenance=false --sbom=false
  4. ECR push → Lambda update-function-code on API + Worker
  5. Post-deploy: GET /api/warmup (verify cold start healthy)
```

> **Lambda build requirement.** Docker BuildKit creates multi-platform manifests with attestations that Lambda rejects. All builds use `--provenance=false --sbom=false` for compatibility.

Base image

The Lambda base image is `public.ecr.aws/lambda/python:3.11` on Amazon Linux 2023. The AL2023 upgrade unlocked DuckDB native httpfs (no more `/tmp` staging) and is a prerequisite for the parquet migration's consumer-cutover stage.

Terraform

Infrastructure changes flow through Terraform with a remote backend — state in `s3://rigacap-prod-terraform-state-149218244179` and locks in DynamoDB `rigacap-prod-terraform-locks`. Plans are run with `AWS_PROFILE=rigacap` against account `149218244179`; the default profile points at a different account and is explicitly never used for production work.

Migration discipline

SQLAlchemy auto-includes every model column in SELECT queries. A column in the model that does not yet exist in the database breaks every query against that table — including auth. The platform follows a migration-first pattern: deploy the migration SQL first via the `run_migration` Lambda event, verify the columns exist, then deploy the model change in a second commit. Schema changes never ship during business hours.

Boring numbers, *watched.*

Current monitoring

COMPONENT	METHOD	DETAILS
Lambda logs	CloudWatch Logs	Python logging, 30-day retention.
API Gateway	CloudWatch metrics	4xx and 5xx error rates per route.
CloudWatch alarms	SNS → email	Lambda errors / throttles / duration; API 5xx and 4xx; RDS CPU / storage / connections; Worker errors and duration.
Pipeline health monitor	HealthMonitorService	Multi-category checks across data freshness, worker performance, database, signals, and infrastructure.
Data fetch tracking	S3 metadata JSON	Source used (Alpaca / yfinance), settlement result, fetch duration, fallback status.
Database	RDS console metrics	CPU, storage, connection counts.

Performance characteristics

OPERATION	LATENCY	RESOURCE
Dashboard load	< 2 seconds	Pre-computed JSON from CloudFront.
Daily signal scan	30–45 seconds	Worker Lambda, ~4,000 symbols, Alpaca SIP.

OPERATION	LATENCY	RESOURCE
Single backtest (90d, 100 symbols)	20–40 seconds	Worker Lambda, 3 GB.
Walk-forward (multi-year)	10–30 minutes	Step Functions fan-out across the Worker Lambda.
AI content generation	2–5 seconds	Claude API (Sonnet 4.5).
Chart card generation	1–3 seconds	matplotlib on the Worker Lambda.

Resource utilization

1 + 3 GB API + WORKER MEMORY	269 MB S3 PICKLE (GZIP)	20 GB RDS STORAGE

Concurrency

Account-level Lambda concurrency: 1,000 total. Reserved concurrency: 50 on the API Lambda, 200 on the Worker Lambda. The Worker reservation was raised from ten after a self-throttling incident demonstrated that scheduled fan-out workloads can collide with on-demand admin invocations. Manual Lambda invocations always target the Worker so the API surface stays available for subscribers.

What the platform *imports.*

Backend — Python 3.11 on Amazon Linux 2023

- **FastAPI** 0.109.0 with Uvicorn 0.27.0 for local dev.
- **Mangum** 0.17.0 — ASGI to Lambda adapter.
- **SQLAlchemy** 2.0.25 with **asyncpg** 0.29.0.
- **Pydantic** 2.5.0 for request and response models.
- **pandas** 2.1.4 with **numpy** 1.26.3 (numpy 2.x is incompatible with the pickled DataFrames).
- **pyarrow** + **duckdb** for the parquet shadow and ad-hoc SQL.
- **alpaca-py** ≥ 0.30.0 for SIP market data.
- **yfinance** ≥ 1.1.0 for fallback bars and index symbols.
- **Optuna** ≥ 3.5.0 for parameter optimization.
- **matplotlib** ≥ 3.8.0 for chart card rendering.
- **boto3** 1.34.0 — AWS SDK.
- **Stripe** 7.10.0 — payments.
- **python-jose** 3.3.0 with **passlib** 1.7.4 for JWT and bcrypt.
- **httpx** 0.26.0 — async HTTP client used for the Claude API and platform graph APIs.
- **APScheduler** 3.10.4 — local-dev scheduler; production uses EventBridge.

Frontend — Node 18

- **React** 18.2.0 · **Vite** 5.0.0 · **TailwindCSS** 3.3.5
- **React Router** 6.20.0 · **Recharts** 2.10.0
- **Axios** 1.6.0 · **Lucide React** 0.292.0

- Puppeteer 24.37.3 · ESLint 8.53.0 · Vitest 0.34.6

Mobile — Expo

- React Native via Expo SDK; OTA updates published to the `preview` channel.
- Expo push notifications service for sell alerts and high-conviction signal events.

External APIs

- Anthropic Claude — `claude-sonnet-4-5` for social drafts, trade autopsies, and editorial copy. Direct httpx POST; the official SDK is intentionally not vendored on Lambda.
- X (Twitter API v2) — tweet publish, contextual reply scanning.
- Threads — `graph.threads.net` publish and reply.
- Instagram-with-Instagram-Login — `graph.instagram.com` publish and comment moderation, IG-native long-lived tokens.
- Stripe — subscriptions, customer portal, coupon-backed referrals.
- Cloudflare Turnstile — bot protection at registration.
- Google & Apple OAuth — identity providers.
- HeyGen — AI avatar video pipeline (in development).
- Expo Push — mobile notification delivery.
